Looping Statements in TypeScript

Looping statements in TypeScript allow executing a block of code multiple times based on a condition. Below are the corrected syntax and examples for each loop.

1. while Loop

The while loop executes a block of code **as long as** the condition is true.

Syntax:

```
while (condition) {
   // Code to execute
}
```

Example:

```
let i: number = 1;
while (i <= 5) {
   console.log(i);
   i++;
}
```

Output:

```
1
2
3
4
5
```

- **The loop checks the condition before execution. If i <= 5, it runs; otherwise, it stops.**

2. do-while Loop

The do-while loop **executes the code at least once** before checking the condition.

Syntax:

```
do {
   // Code to execute
} while (condition);
```

Example:

```
let j: number = 1;
do {
   console.log(j);
   j++;
} while (j <= 5);
```

Output:

```
1
2
3
4
5
```

- **Executes once even if the condition is false.**

3. for Loop

The for loop is useful when the number of iterations is known.

Syntax:

```
for (initialization; condition; increment/decrement) {
   // Code to execute
}
```

Example:

```
for (let k: number = 1; k <= 5; k++) {
   console.log(k);
}
```

Output:

1

2

3

4

5

- **Includes initialization, condition check, and increment/decrement in a single line.**

4. break Statement

The break statement **stops the loop immediately** when a condition is met.

Syntax:

```
for (initialization; condition; increment/decrement) {
   if (someCondition) {
      break;
   }
   // Code to execute
}
```

Example:

```
for (let n: number = 1; n <= 10; n++) {
   if (n === 5) {
      break; // Exits the loop when n is 5
   }
   console.log(n);
}
```

Output:

1

2

3

4

🔹 **The loop exits when n reaches 5.**

5. continue Statement

The continue statement **skips the current iteration** and moves to the next one.

Syntax:

```
for (initialization; condition; increment/decrement) {
   if (someCondition) {
      continue;
   }
   // Code to execute
}
```

Example:

```
for (let m: number = 1; m <= 5; m++) {
   if (m === 3) {
      continue; // Skips when m is 3
   }
   console.log(m);
}
```

Output:

```
1
2
4
5
```

🔹 **Skips 3 and continues with the next iteration.**

6. for-of Loop (Array Iteration)

The for-of loop is used to iterate over the **values** of arrays (or iterable objects).

Syntax:

```
for (let variable of iterable) {
```

```
    // Code to execute
}
```

Example:

```
let x = ["Apple", "Banana", "Mango"];
for (let y of x) {
    console.log(y);
}
```

Output:

```
Apple
Banana
Mango
```

- **Used to iterate values of arrays (or iterable objects).**
- **Clean, simple syntax.**

7. for-in Loop (Arrays or Object Iteration)

The for-in loop is used to iterate over the **keys** of an object. With arrays, it returns the **indexes**; with objects, it returns the **keys**.

Syntax:

```
for (let key in object) {
    // Code to execute
}
```

Array Example:

```
let x = ["Nag", "Kalam"];
for (let y in x) {
    console.log(y + ": " + x[y]);
}
```

Output:

```
0: Nag
1: Kalam
```

Object Example:

```
let x = { name: "Nag", age: 25, course: "Playwright" };
for (let y in x) {
   console.log(y + ": " + x[y]);
}
```

Output:

```
name: Nag
age: 25
course: Playwright
```

- **Used to iterate keys of an object.**
- **With arrays, it returns indexes; with objects, it returns keys.**

Comparison: while Loop vs do-while Loop

Feature	while Loop	do-while Loop
Condition Check	Before execution	After first execution
Execution Guarantee	May not execute if the condition is false initially	Executes at least once

Example Demonstration:

```
let x: number = 5;

while (x < 5) {
   console.log("Inside while loop"); // Won't run because x is not < 5
}


do {
   console.log("Inside do-while loop"); // Runs once before checking condition
} while (x < 5);
```

Output:

```
Inside do-while loop
```

Conclusion

- **Use while** when you want to check the condition before execution.
- **Use do-while** when you need the loop to run at least once.
- **Use for** when you know how many times the loop should run.
- **Use break** to stop a loop early.
- **Use continue** to skip an iteration and proceed to the next one.